AICA CAPSTONE PROJECT: DATA ENGINEERING TRACK (2025/2026 COHORT 2)

Building a Maintainable ETL and ELT Pipeline for Weather Analytics Using API Data, SQL, Star Schema, and Airflow

==========

DATASET TO USE

Data extracted from a public API:

Open-Meteo Weather API
https://open-meteo.com/

API Documentation:
https://open-meteo.com/en/docs

==========

PROJECT SCENARIO

You are working as a Junior Data Engineer for a weather analytics company. The company collects weather data from external APIs and uses it to support reporting, forecasting, and business decision-making.

The company requires an automated data pipeline that extracts data from an API, transforms and validates the data, loads it into a SQL database using a star schema design, and runs automatically every day.

Your task is to build a maintainable Python data pipeline that prepares the data for analytics and reporting.

==========

LEARNING OBJECTIVES

By completing this project, students should demonstrate that they can:

1. Build an ETL pipeline in Python
2. Build a simple ELT workflow
3. Extract data from an API

4. Apply software engineering principles
5. Use modular code
6. Create reusable functions/classes
7. Add logging and exception handling
8. Validate data quality
9. Write unit tests
10. Design a star schema
11. Load data into a SQL database
12. Automate a data pipeline using Airflow
13. Document their work properly
14. Publish a project on GitHub

==========

PART A: ETL PIPELINE

Students must build a pipeline that performs the following:

1. EXTRACT

Extract data from the Open-Meteo Weather API.

The extraction code should:

• Connect to the API successfully
• Retrieve data using Python requests
• Validate API responses
• Log successful extraction
• Handle API connection errors properly
• Handle invalid responses properly

---

2. TRANSFORM

Clean and transform the extracted data.

Required Transformations

1. Clean column names
2. Convert date and timestamp fields to proper datetime formats
3. Convert numeric fields to appropriate data types
4. Handle missing values in important fields
5. Remove duplicate records if any

6. Validate weather measurements
7. Create derived fields where necessary
8. Standardize location names
9. Validate that required columns are present
10. Prepare the data for loading into fact and dimension tables

---

3. LOAD

Load the transformed data into a SQL database.

The load function should:

• Establish a database connection
• Create tables if they do not exist
• Load data into fact and dimension tables
• Log successful loads
• Handle database connection errors
• Handle data loading errors

==========

PART B: ELT WORKFLOW

Students should also implement a simple ELT version.

In ELT:

1. Extract data from the API
2. Load the raw data into a staging table
3. Transform the staged data inside the pipeline
4. Load the transformed data into the final analytical tables

==========

PART C: STAR SCHEMA DESIGN

Students must design and implement a star schema.

The schema should contain:

Fact Table and Dimension Tables

Students should clearly document their schema design and explain the relationship between fact and dimension tables.

==========

PART D: AIRFLOW AUTOMATION

Students must automate the pipeline using Apache Airflow.

The Airflow workflow should:

1. Extract data from the API
2. Transform the data
3. Validate the data
4. Load data into the database

The pipeline should be configured to run daily.

Students should provide:

• Airflow DAG file
• DAG screenshot showing successful execution
• Explanation of task dependencies

==========

PART E: SOFTWARE ENGINEERING REQUIREMENTS

The project must include:

1. MODULAR CODE

Separate extraction, transformation, loading, validation, and utility functions into different modules.

2. CLASS-BASED PIPELINE

Create a reusable pipeline class.

3. LOGGING

Log major pipeline events and errors.

4. EXCEPTION HANDLING

Handle API, database, file, and validation errors appropriately.

5. DOCUMENTATION

Provide a detailed README file.

6. UNIT TESTING

Write unit tests using pytest.

7. CODE QUALITY

Follow PEP 8 guidelines and maintain clean, readable code.

==========

DELIVERABLES

Students should submit a GitHub repository link containing:

1. Full project folder
2. Airflow DAG file
3. SQL schema file
4. Log file
5. README.md
6. requirements.txt
7. pytest test files
8. Screenshots of successful Airflow execution
9. Sample output from the SQL database

==========

FINAL INSTRUCTION TO STUDENTS

Your goal is not only to move data from one place to another.

Your goal is to build a production-style data pipeline that another developer can understand, run, maintain, automate, and improve.

The project should demonstrate:

• Good data engineering practices
• Software engineering principles

- Database design skills
- Data validation techniques
- Testing methodologies
- Documentation standards
- Pipeline automation using Apache Airflow

Build your solution as if it will be handed over to another data engineering team for long-term maintenance and future enhancements.